INDEX

Computer Networks

NAME : A. Deiva Lakshmee STD.: AIDS SEC.: III yr. ROLL NO.: 47 SUB.: Computer Networks

S. No.	Date	Title	Page No.	Teacher's Sign/Remarks
1	16/07/26	Study of Various commands used in linux & Windows		Sree
2	30/7/26	CISCO packet tracer simulator tools		Sree
3	~~08~~30/7/26	Experiment on cisco packet tracer		Sree
4	01/08/26	Setup and configure a LAN using a switch and ethernet cables		Sree
5	06/08/26	Packet cable tool : Wireshark		Sree
6	13/08/26	Flow control at data link layer using sliding window protocol		Sree
7	13/08/26	Error detection & Correction using Hamming Code		Sree
8	20/08/26	NMAP to discover live host Using Nmap Scan		Sree
9	22/08/26	Implementation of Subnetting in cisco packet		Sree
10	27/08/26	Interpetworking routers in cisco packet simulator		Sree
11	29/08/26	Routing a network layer		Sree
12	10/09/26	End to End Communication at transport layer		Sree
13	12/09/26	Implementing an ping program		Sree
14	17/09/26	Packet sniffing Using Raw Sockets		Sree
15	24/09/26	Weblog Analysis Using Webalizer		Sree

completed

Ex: NO: 1

16/07/24

Study of various Networks commands used in Linux and windows

Aim :-

To study and practice various networking commands used in Linux and windows for networking configuration, trouble shooting and diagnostics.

Commands :-

1- ARP (Address Resolution Protocol)

- Displays and manages the ARP table, which maps IP address to MAC Address.

```
C:\Windows\system32>arp

Displays and modifies the IP-to-Physical address translation tables used by
address resolution protocol (ARP).

ARP -s inet_addr eth_addr [if_addr]
ARP -d inet_addr [if_addr]
ARP -a [inet_addr] [-N if_addr] [-v]

  -a            Displays current ARP entries by interrogating the current
                protocol data.  If inet_addr is specified, the IP and Physical
                addresses for only the specified computer are displayed.  If
                more than one network interface uses ARP, entries for each ARP
                table are displayed.
  -g            Same as -a.
  -v            Displays current ARP entries in verbose mode.  All invalid
                entries and entries on the loop-back interface will be shown.
  inet_addr     Specifies an internet address.
  -N if_addr    Displays the ARP entries for the network interface specified
                by if_addr.
  -d            Deletes the host specified by inet_addr. inet_addr may be
                wildcarded with * to delete all hosts.
  -s            Adds the host and associates the Internet address inet_addr
                with the Physical address eth_addr.  The Physical address is
                given as 6 hexadecimal bytes separated by hyphens. The entry
                is permanent.
  eth_addr      Specifies a physical address.
  if_addr       If present, this specifies the Internet address of the
                interface whose address translation table should be modified.
                If not present, the first applicable interface will be used.

Example:
  > arp -s 157.55.85.212   00-aa-00-62-c6-09  .... Adds a static entry.
  > arp -a                                    .... Displays the arp table.

C:\Windows\system32>
```

2] ~~Hostname~~ IP Config

```
C:\Windows\system32>ipconfig

Windows IP Configuration


Ethernet adapter Npcap Loopback Adapter:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::b923:c6f7:7122:8700%6
   Autoconfiguration IPv4 Address. . : 169.254.158.128
   Subnet Mask . . . . . : 255.255.0.0
   Default Gateway . . . . . :
```

```
Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::390c:dc1c:3ca7:94bc%3
   IPv4 Address. . . . . : 192.168.56.1
   Subnet Mask . . . . . : 255.255.255.0
   Default Gateway . . . . . :
```

2. Host Name:

Displays or changes the name of the local computer

Output : Ananthideiva

3) IP config (windows) / if config (Linux)

Displays and configures the IP address subnet mask, and default gateway

```
Ethernet adapter Ethernet:

   Connection-specific DNS Suffix  . :
   IPv4 Address. . . . . : 172.16.10.86
   Subnet Mask . . . . . : 255.255.252.0
   Default Gateway . . . . . : 172.16.8.1

C:\Windows\system32>
```

4] NBTStat

```
C:\Windows\system32>nbtstat

Displays protocol statistics and current TCP/IP connections using NBT
(NetBIOS over TCP/IP).

NBTSTAT [ [-a RemoteName] [-A IP address] [-c] [-n]
        [-r] [-R] [-RR] [-s] [-S] [interval] ]

  -a   (adapter status) Lists the remote machine's name table given its name
  -A   (Adapter status) Lists the remote machine's name table given its
                        IP address.
  -c   (cache)          Lists NBT's cache of remote [machine] names and their IP addresses
  -n   (names)          Lists local NetBIOS names.
  -r   (resolved)       Lists names resolved by broadcast and via WINS
  -R   (Reload)         Purges and reloads the remote cache name table
  -S   (Sessions)       Lists sessions table with the destination IP addresses
  -s   (sessions)       Lists sessions table converting destination IP
                        addresses to computer NETBIOS names.
  -RR  (ReleaseRefresh) Sends Name Release packets to WINS and then, starts Refresh

  RemoteName   Remote host machine name.
  IP address   Dotted decimal representation of the IP address.
  interval     Redisplays selected statistics, pausing interval seconds
               between each display. Press Ctrl+C to stop redisplaying
               statistics.

C:\Windows\system32>
```

4) NBTStat (Net BIOS statics).

Displays NetBIOS over Tcp/Ip protocol statistics and NetBIOS name tables.

[Image on left side]

5) Netstat

Displays active network connections, routing tables and listening ports.

```
C:\Windows\system32>netstat

Active Connections

  Proto  Local Address          Foreign Address        State
  TCP    127.0.0.1:49675        DESKTOP-G0LVE4J:49676  ESTABLISHED
  TCP    127.0.0.1:49676        DESKTOP-G0LVE4J:49675  ESTABLISHED
  TCP    127.0.0.1:49677        DESKTOP-G0LVE4J:49678  ESTABLISHED
  TCP    127.0.0.1:49678        DESKTOP-G0LVE4J:49677  ESTABLISHED
  TCP    172.16.10.86:13133     4.213.25.242:https     ESTABLISHED
  TCP    172.16.10.86:13199     lcbomp-in-f188:5228    ESTABLISHED
  TCP    172.16.10.86:13258     a23-202-229-23:https   CLOSE_WAIT
  TCP    172.16.10.86:13259     150.171.110.82:https   CLOSE_WAIT
  TCP    172.16.10.86:13260     131.253.33.254:https   CLOSE_WAIT
  TCP    172.16.10.86:13261     135.233.41.222:https   CLOSE_WAIT
  TCP    172.16.10.86:13262     204.79.197.222:https   CLOSE_WAIT
  TCP    172.16.10.86:13270     72.154.187.226:https   CLOSE_WAIT
  TCP    172.16.10.86:13272     20.100.121.19:https    CLOSE_WAIT
  TCP    172.16.10.86:13273     a-0003:http            CLOSE_WAIT
  TCP    172.16.10.86:13278     13.107.42.254:https    CLOSE_WAIT
  TCP    172.16.10.86:13279     52.108.9.254:https     CLOSE_WAIT
  TCP    172.16.10.86:13281     172.202.65.254:https   CLOSE_WAIT
  TCP    172.16.10.86:13293     a23-211-142-112:https  TIME_WAIT
  TCP    172.16.10.86:13294     a23-211-142-112:https  TIME_WAIT
  TCP    172.16.10.86:13295     a23-211-142-112:https  TIME_WAIT
  TCP    172.16.10.86:13297     bkk02s04-in-f14:https  ESTABLISHED
  TCP    172.16.10.86:13298     4.213.25.242:https     ESTABLISHED
  TCP    [fe80::390c:dc1c:3ca7:94bc%3]:1521  DESKTOP-G0LVE4J:49679  ESTABLISHED
  TCP    [fe80::390c:dc1c:3ca7:94bc%3]:49679  DESKTOP-G0LVE4J:1521  ESTABLISHED

C:\Windows\system32>
```

DOMS | Page No.
Date / /

6. NS Lookup

Queries DNS servers to find the IP address or domain name of a host.

```
C:\Windows\system32>nslookup www.google.com
Server:  UnKnown
Address:  172.16.8.1

Non-authoritative answer:
Name:    www.google.com
Addresses:  2001:4860:4827:7700::
          2001:4860:482b:7700::
          2001:4860:4828:7700::
          2001:4860:482a:7700::
          2001:4860:4826:7700::
          2001:4860:4829:7700::
          2001:4860:482d:7700::
          2001:4860:482c:7700::
          142.251.156.119
          142.251.155.119
          142.251.151.119
          142.251.150.119
          142.251.157.119
          142.251.152.119
          142.251.154.119
          142.251.153.119

C:\Windows\system32>
```

7. Path Ping:

Combines the features of ping and Traceroute to identify packet loss and network latency

```
C:\Windows\system32>pathping

Usage: pathping [-g host-list] [-h maximum_hops] [-i address] [-n]
                [-p period] [-q num_queries] [-w timeout]
                [-4] [-6] target_name

Options:
    -g host-list      Loose source route along host-list.
    -h maximum_hops   Maximum number of hops to search for target.
    -i address        Use the specified source address.
    -n                Do not resolve addresses to hostnames.
    -p period         Wait period milliseconds between pings.
    -q num_queries    Number of queries per hop.
    -w timeout        Wait timeout milliseconds for each reply.
    -4                Force using IPv4.
    -6                Force using IPv6.

C:\Windows\system32>_
```

```
C:\Windows\system32>route

Manipulates network routing tables.

ROUTE [-f] [-p] [-4|-6] command [destination]
                  [MASK netmask]  [gateway] [METRIC metric]  [IF interface]

  -f            Clears the routing tables of all gateway entries.  If this is
                used in conjunction with one of the commands, the tables are
                cleared prior to running the command.

  -p            When used with the ADD command, makes a route persistent across
                boots of the system. By default, routes are not preserved
                when the system is restarted. Ignored for all other commands,
                which always affect the appropriate persistent routes.

  -4            Force using IPv4.

  -6            Force using IPv6.

  command       One of these:
                  PRINT     Prints  a route
                  ADD       Adds    a route
                  DELETE    Deletes a route
                  CHANGE    Modifies an existing route
  destination   Specifies the host.
  MASK          Specifies that the next parameter is the 'netmask' value.
  netmask       Specifies a subnet mask value for this route entry.
                If not specified, it defaults to 255.255.255.255.
  gateway       Specifies gateway.
  interface     the interface number for the specified route.
  METRIC        specifies the metric, ie. cost for the destination.

All symbolic names used for destination are looked up in the network database
file NETWORKS. The symbolic names for gateway are looked up in the host name
database file HOSTS.

If the command is PRINT or DELETE. Destination or gateway can be a wildcard,
(wildcard is specified as a star '*'), or the gateway argument may be omitted.

If Dest contains a * or ?, it is treated as a shell pattern, and only
matching destination routes are printed. The '*' matches any string,
and '?' matches any one char. Examples: 157.*.1, 157.*, 127.*, *224*.

Pattern match is only allowed in PRINT command.
Diagnostic Notes:
    Invalid MASK generates an error, that is when (DEST & MASK) != DEST.
    Example> route ADD 157.0.0.0 MASK 155.0.0.0 157.55.80.1 IF 1
             The route addition failed: The specified mask parameter is invalid. (Destination & Mask) != Destination.

Examples:

    > route PRINT
    > route PRINT -4
    > route PRINT -6
    > route PRINT 157*                .... Only prints those matching 157*

    > route ADD 157.0.0.0 MASK 255.0.0.0  157.55.80.1 METRIC 3 IF 2
              destination^    ^mask      ^gateway     metric^    ^
                                                       Interface^
    If IF is not given, it tries to find the best interface for a given
    gateway.
    > route ADD 3ffe::/32 3ffe::1

    > route CHANGE 157.0.0.0 MASK 255.0.0.0 157.55.80.5 METRIC 2 IF 2

    CHANGE is used to modify gateway and/or metric only.

    > route DELETE 157.0.0.0
    > route DELETE 3ffe::/32

C:\Windows\system32>_
```

8 Ping (Packet Internet Groper)

Tests network connectivity between two devices by sending ICMP echo requests.

```
C:\Windows\system32>ping

Usage: ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS]
            [-r count] [-s count] [[-j host-list] | [-k host-list]]
            [-w timeout] [-R] [-S srcaddr] [-c compartment] [-p]
            [-4] [-6] target_name

Options:
    -t             Ping the specified host until stopped.
                   To see statistics and continue - type Control-Break;
                   To stop - type Control-C.
    -a             Resolve addresses to hostnames.
    -n count       Number of echo requests to send.
    -l size        Send buffer size.
    -f             Set Don't Fragment flag in packet (IPv4-only).
    -i TTL         Time To Live.
    -v TOS         Type Of Service (IPv4-only. This setting has been deprecated
                   and has no effect on the type of service field in the IP
                   Header).
    -r count       Record route for count hops (IPv4-only).
    -s count       Timestamp for count hops (IPv4-only).
    -j host-list   Loose source route along host-list (IPv4-only).
    -k host-list   Strict source route along host-list (IPv4-only).
    -w timeout     Timeout in milliseconds to wait for each reply.
    -R             Use routing header to test reverse route also (IPv6-only).
                   Per RFC 5095 the use of this routing header has been
                   deprecated. Some systems may drop echo requests if
                   this header is used.
    -S srcaddr     Source address to use.
    -c compartment Routing compartment identifier.
    -p             Ping a Hyper-V Network Virtualization provider address.
    -4             Force using IPv4.
    -6             Force using IPv6.
```

9. Route :

Displays or modifies the IP routing tables used for packet forwarding.

Result:

The networking commands were successfully executed and studied. Their outputs were observed and used to understand network config, connectivity testing, DNS resolution, routing and troubleshooting.

Ex: NO: 3

30/7/26

Experiment on CISCO PACKET TRACER Simulation Tools.

Aim:

To study the Cisco Packet Tracer installation and user interface and understand its environment for designing a simple network

Introduction:

Packet Tracer is a Network simulation and modeling tool used to design and test networks without physical devices.

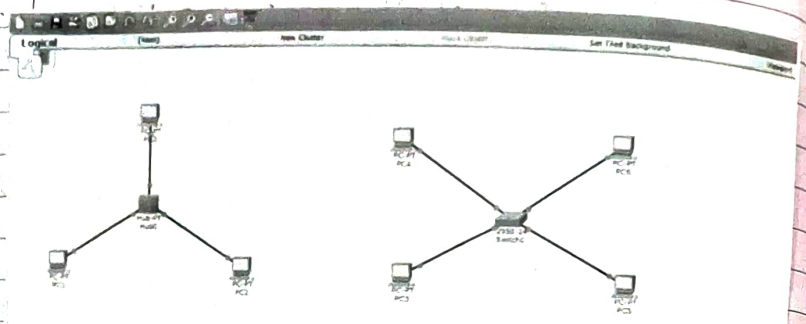
1. Simulates network devices & environments
2. Helps practice network configuration and troubleshooting.
3. Available for windows, Linux, Android and iOS.
4. Protocols behave similarly to real cisco hardware.

Installing Packet Tracer:

Download packet Tracer from http://www.netacad.com using your cisco networking Academy account.

User Interface Overview:

1. Menu bar - Open, SAVE, Print, preferences etc.
2. Main Toolbar - Shortcuts for Open, save, zoom undo, redo & networking Informatics



4) Workspace - Area for creating topologies and viewing simulations

5) Common Tools Bar - Select, move, note, delete inspect, resize & add PDU

6) Real time/Simulation Tabs - Switch between real time & Simulation modes Control packet flow.

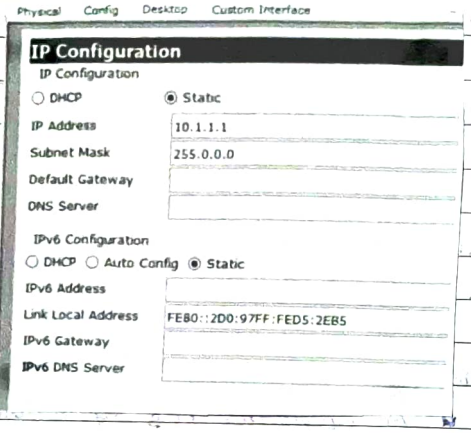
7) Network component Box.

* Device - Type selection - Select device Categories
* Device - Specific selection - Displays model of Selected Categories

8) User → created packet Box - Create and view custom test packets.

Analyse the behaviour of network devices using CISCO PACKET TRACER simulator.

1) Drag & drop:
   * A Generic PCs & 1 Hub
   * A Generic PCs & 1 Switch
2) Select connections:
   * Choose copper straight-Trough cable
   * Connect each PC to the Hub. Ensure the link LED turns green
   * Similarly, connect the four pcs to the Switch.
3) Configure IP addresses:
   * Open each PC → Desktop → IP Configuration
   * Assign an IP address & Subnet mask.
   * Default Gateway and DNS are not required.
4) click Add Simple PDU (Message icon)

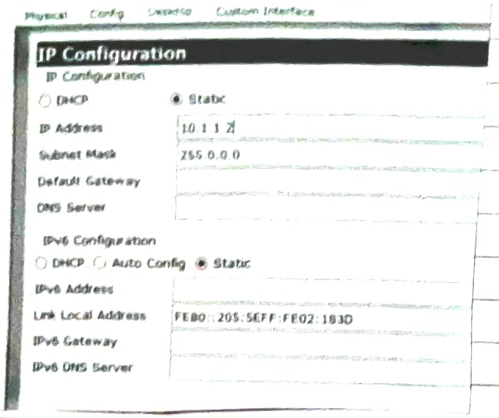


* select the source PC and then the destination PC connected to the Hub.

5) In Realtime mode, observe the PDU transmission.

6) Repeat the same steps for the PCs connected to the switch.

7) Compare how the hub & switch forward the PDU, then write your observations and conclusion.



Result:

Thus the packet tracer tool installation and user interface overview has studied and done successfully.

Ex: NO: 2
30.07.2026

Study of Different types of Network cables.

Aim:

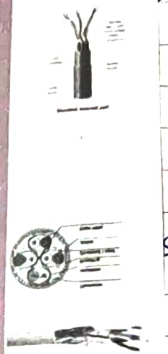
To study the different types of Network cables.

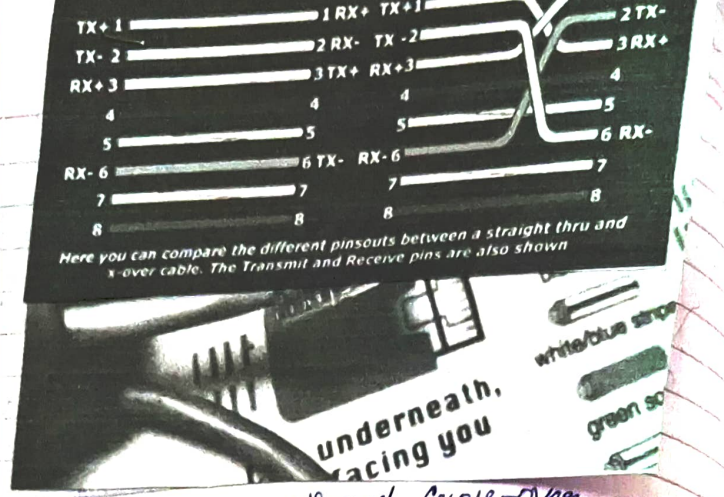
a) Understand different types of network cable.

Different type of cables used in networking are

1. Unshielded Twisted pair (UTP) cable.
2. Shielded Twisted pair (STP) Cable
3. Coaxial cable.
4. Fibre optic cable.

Cable type	Category	Maximum Data Transmission	Advantages/ Disadvantages	Application Use
UTP	Category 3	10 bps	Advantages:- → Cheaper in cost → Easy to install as they have a Smaller overall diameter. Disadvantages: → More prone to (EMI) Electromagnetic interference and noise	→ 10 Base T ethernet
	Category 5	up to 100Mbps		→ Fast Ethernet
	Category 5e	1 GbPS		→ Gigabit Ethernet
STP	Category 6 / 6a	10 Gbps	Advantages:- → Shielded. → Faster than UTP → less susceptible to noise and interface	→ Gigabit Ethernet → 10G Ethernet (55m) → Widely used in data centres.
STP	Category 7	10Gbps	Disadvantages: → Expensive → Greater installation effort.	→ Gigabit ethernet. → 10G Ethernet (100 m)
Coaxial Cable	RG-6 / RG-59 / RG-11	10-100Mbps	Advantages:- → High bandwidth → immune to interface → Low loss bandwidth → Versatile Disadvantages:- → Limited distance → Cost → Size is bulky.	→ Speed of Signal is 500m → Television network. → High Speed internet connections.
Fibre optics Cable	Single mode / Multimode	100 Gbps	Advantages:- → High Speed → High bandwidth → High security → Long distance Disadvantages:- → Expensive → Requires skilled installers	→ Maximum distance of fibre optics cable is around 100 meters.





b) Make your own Ethernet Cross-Over Cable/ straight cable.

Tools and parts needed:-

→ Ethernet Cabling - CAT5e is certified for gigabit support, but CAT5 cabling works as well, just over shorter distances.
→ A crimping tool. This is an all-in one networking tool shaped to push down the pins in the plug and strip and cut the shielding off the cables.
→ Two RJ45 plugs.
→ Optional two plugs shields.

Step 1:

To start construction of the device, begin by threading shields onto the cable

Step 2:

Next, strip approximately 1.5 cm of cable shielding from both ends. The crimping tool has a round area to complete this task.

Step 3:

After you will need to untangle the wires; there should be four "twisted pairs". Referencing back to the sheet, arrange them from top to bottom. One end should be in arrangement A and the other in B.

Step 4:

Once the order is correct, bunch them together in a line, and if there are any that stick out farther than others, snip them back to create an even level, the difficult aspect is placing these into the RJ45 plug without messing up the order. To do so, hold the plug with the clip side facing away from you and have the gold pins facing towards you as shown

Step 5:

Next, push the cable right in. The notch at the end of the plug needs to be just over the cable shielding and if it isn't, that means that you stripped off too much shielding. Simply snip the cables back a little more.

Step 6:

After the wires are securely sitting inside the plug, insert it into the crimping tool and push down. It should be shaped correctly, but pushing too hard can crack the fragile plastic plug.

Step 7:

Lastly, repeat for the other end using diagram B (to make a crossover cable) / using diagram A (to make a straight through cable.) To test it, plug it in and attempt to connect two devices directly

Result:

Thus, the different types of network cables and ethernet cross-over cable / straight cables were studied.

EX. NO: 1
01.08.26

Setup and Configure a LAN (Local Area network) using a switch and ethernet cables.

Aim:

Setup and configure a LAN (local Area network) using a switch and ethernet cables in your lab.

What is a LAN?

A local Area Network (LAN) refers to a network that connects devices within a limited area, such as an office building, a school, or home. It enables users to share resources, including data, printers, and internet access. LAN connects devices to promote collaborations and transfer information between users, such as computers, printers, servers and switches.

How to set up a LAN

Step 1:

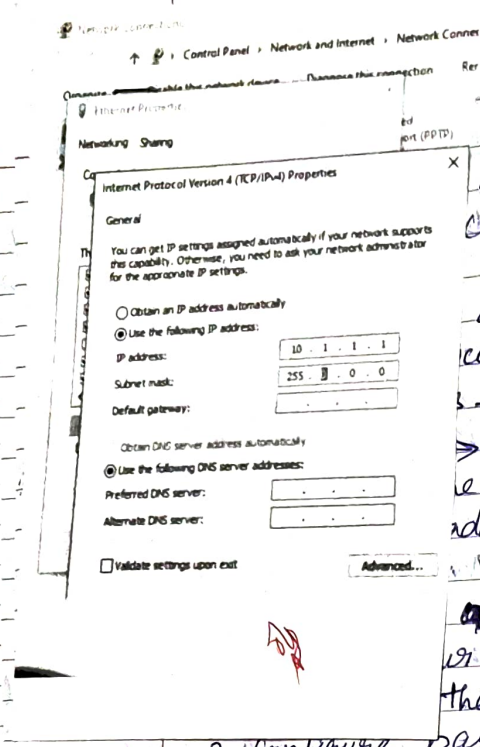
Plan and design an appropriate network topology taking into account network requirements and equipment location

Step 2:

You can take 4 computers, a switch with 8, 16 (or) 24 ports which is sufficient for networks of these sizes and 4 ethernet cables

Step 3:

Connect your computers to network switch via an ethernet cable, which is as simple as plugging one end of the ethernet cable into your network switch.



address to your pc's
client computer as admini

and Internet connections
ical Area connection (Etherne
→ select Internet protocol
→ click on properties →
e following ip addreis opti
ddress.

Network switch.
r computer to the switch
the web interface.

3. Configure basic settings.

4. Assign Ip. address as : 10.1.1.5, Subnet mask 255.0.0.0

Step 6:

Check the connectivity between switch and other machine by using ping command in the Command prompt of the device.

Step 7:

- Select a folder → go to properties → click sharing tab → share it with everyone on the same LAN

Step 8:

Try to access the shared folder from others computers of the network.

Result:

Thus the setup and configure a LAN using switch & Ethernet cable was successfully completed.

EX: NO: 5
01.08.26

Experiment on packet capture tool : Wireshark.

Aim:

Experiments on packet capture tool : wireshark.

packet sniffer.

→ Sniffs messages being sent / received from by your computer.

→ Passive program.
- never sends packets itself
- no packets addressed to it
- receives a copy of all packets (sent/recieved)

packet sniffer structure Diagnostic Tools.

→ Tcp dump
- Eg: tcpdump -enx host 10.129.41.2 -w exe3.out

→ Wireshark -r exe3.out

Theory:

→ WireShark is a network packet analyzer used to capture and inspect network traffic in real time

→ It helps in troubleshooting network issues analyzing protocols, monitoring traffic, and learning network communication

→ It captures packets, decodes protocols, applies filters, displays statistics, and analyzes packet flow

Features:

→ Captures live network traffic
→ Supports protocols decoding.
→ Provides capture and display filters.
→ Displays packet details and statistics

→ Generates flow graphs.
→ Saves captured packets for later analysis

Wireshark Interface:
1) Packet List - Displays all captured packets
2) Packet Details - Shows protocol information for the selected packet
3) Packet Bytes - Displays packet data in hexadecimal format.

Common Filters:
→ tcp - Displays Tcp packets
→ udp - Displays UDP packets
→ arp - Displays ARP packets
→ dns - Displays DNS packets
→ http - Displays HTTP packets
→ icmp - Displays ICMP packets
→ dhcp - Displays DHCP packets

Procedure:
1) Open wireshark
2) Select the Ethernet / Local Area connection interface
3) Go to Capture → options
4) Set Stop capture after 100 packets
5) Click Start to begin packet capture
6) Apply the required filter (Tcp, UDP, ARP, DNS, HTTP, ICMP, DHCP)
7) Inspect the captured packets
8) For Tcp and DNS, View statistics → Flow Graph
9) Save the captured packets

TCP and UDP packets:

→ Applied the tcp and udp display fillers in wireshark to view only TCP and UDP packets.
→ Observed communication between the source and destination using IP addresses and port numbers.
→ Tcp packets showed reliable communication with connection establishment and acknowledgements, while UDP packets showed fast, connectionless communication
→ Examined packets details such as source/destination address, ports, protocol headers and the payloads
→ Viewed the communication sequence using statistics → Flow Graph.
→ Sucessfully analyzed and saved the capture TCP/UDP packets

Result:
Successfully captured, filtered, and analysed different network packets using wireShark. The tcp and udp packets helped understand the communication between network devices and network protocols.

Ex: No: 6
13/08/26

Flow Control at Data Link Layer Using Sliding Window Protocol.

Aim:

To write a program to implement flow control at the Data Link Layer using the Sliding Window Protocol, and to simulate the flow of frames from one node to another.

Algorithm:

1. Start the program
2. Read the total number of frames to be sent (N), the sender's window size (W), and optionally a frame number to simulate as lost during transmission.
3. Initialize base = 0 (Sequence number of the oldest unacknowledged frame) and nextSeq = 0 (Sequence number of the next frame to be sent)
4. Repeat while base < N:
   a. Determine the current sender window as the range [base, base+W-1].
   b. Send all frames with sequence numbers from nextSeq up to (base+W-1), incrementing nextSeq after each frame, since the window restricts the sender to at most W outstanding (unacknowledged) frames at any time.
5. c. If the lost frame lie within the frames just sent:
      i) The receiver discards the lost frame and any subsequent out-of-order frames.
      ii) The sender times out and retransmits all frames starting from the lost frame (Go-Back-N)
   d. Else (no loss in this batch):
      i) The receiver accepts each frame in order and sends a cumulative acknowledgement (ACK) for it

ii) On receiving the ACK for the last frame sent, the sender slides its window forward:
base = nextSeq.
5. Repeat Step 4 until all N frames have been transmitted and acknowledged.
6. Stop the program

PROGRAM:

```
def simulate(total_frames, window_size, lost_frame):
    base = 0
    next_seq = 0
    while base < total_frames:
        window_end = min(base + window_size - 1, total_frames - 1)
        print("\n.....")
        print(f'Sender Window: [Frame {base} - Frame {window_end}]")
        # Send all frames within the current window that haven't been sent
        while next_seq < base + window_size and next_seq < total_frames:
            print(f'Node A --> Sending Frame {next_seq}")
            if next_seq == lost_frame:
                print(f"    (Frame {next_seq} LOST in transit)
            next_seq += 1
        if lost_frame is not None and base <= lost_frame < next_seq:
            # a frame in this batch was lost: receiver discards it and
            # everything after it (Go-Back-N); Sender times out and
            # resends starting from the lost frame
            print(f'Node B --> Frame {lost_frame} missing. Discarding out-of-order
```

```
print(f'Node A -> Timeout! Resending window
    starting from Frame {lost_frame}.")
next_seq = lost_frame
lost_frame = None # loss already handled;
        retransmission succeeds continue.

# no loss in this batch: receiver acknowledges
        every frame in order
for f in range (base, next_seq):
    print(f'Node B -> Received Frame {f},
        Sending ACK {f}")
print(f'Node A -> ACK {next_seq-1} received.
        Sliding window forward.")
base = next_seq
print("\n.....")
print(f'All {total_frames} frames successfully
    transmitted from Node A to Node B".)

def main():
    print("===SLIDING WINDOW PROTOCOL-FLOW
        CONTROL SIMULATION (Go Back N)===\n")
    total_frames = int(input("Enter total number of
        frames to send :"))
    window_size = int(input('Ender sender's
        window size :'))
    lost_frame = int(input(f'Enter frame number
        to simulate as LOST \n(0 to {total_frames}:)
    if lost_frame == 0:
        lost_frame = None
    simulate(total_frames, window_size, lost_frame)

if __name__ == "__main__":
    main()
```

DOMS Page No.
Date / /

OUTPUT:

===SLIDING WINDOW PROTOCOL - FLOW CONTROL SIMULATION (Go-Back-N)===

Enter total number of frames to send : 8
Enter Senders window Size : 4
Enter frame number to simulate as lost (0 to 7, enter -1 for no loss) : 5

Sender Window : [Frame 0 - Frame 3]
Node A --> Sending Frame 0
Node A --> Sending Frame 1
Node A --> Sending Frame 2
Node A --> Sending Frame 3
Node B --> Received Frame 0, sending ACK 0
Node B --> Received Frame 1, sending ACK 1
Node B --> Received Frame 2, sending ACK 2
Node B --> Received Frame 3, sending ACK 3
Node A --> ACK 3 received. Sliding Window forward.

Sender window : [Frame 4 - Frame 7]
Node A --> Sending Frame 4
Node A --> Sending Frame 5
Frame 5 lost in Transit!)
Node A --> Sending Frame 6
Node A --> Sending Frame 7
Node B --> Frame 5 missing. Discarding out of order frames
Node A --> Time Out! Resending window starting from Frame 5.

Sender Window : [Frame 4 - Frame 7]
Node A ...> sending Frame 5
Node A ...> Sending Frame 6
Node A ...> sending Frame 7.
Node B ...> Received Frame 4, Sending ACK 4
Node B --> Received Frame 5. Sending ACK 5.

Node B --> Received Frame 6 Sending ACK 6
Node B --> Received Frame 7, Sending ACK 7
Node A --> ACK 7 received. Sliding Window forward.

All 8 frames successfully transmitted from Node A to Node B.

RESULT:

The program to implement flow control at the Data Link Layer using the Sliding Window Protocol was successfully written in Python, executed and verified by simulation frames between two nodes (Node A as sender and Node B as receiver).

Ex. No: 7
31/08/24

Error Detection and Correction Using Hamming code.

AIM:

To write a program to implement error detection and correction using the Hamming Code

ALGORITHM:

1. Start the program.
2. Read the number of 4-bit data blocks (n) forming the input data stream
3. For each 4-bit data block D = $d_1\ d_2\ d_3\ d_4$, perform Hamming encoding.
   a. Arrange 7 bits with parity bits at positions 1, 2, and 4 and data bits at positions 3, 5, 6, and 7.
   b. Compute the parity bits using even parity.
      $P1 = d1$ XOR $d2$ XOR $d4$
      $P2 = d1$ XOR $d3$ XOR $d4$
   c. Form the 7-bit Hamming code word: $P1\ P2\ d1\ P4\ d2\ d3\ d4$.
4. Simulate transmission by introducing a single-bit error at a chosen bit position
5. At the receiver, recompute the check bits (syndrome) from the received code word.
   $C_1 = P1$ XOR $d1$ XOR $d2$ XOR $d4$
   $C_2 = P2$ XOR $d1$ XOR $d3$ XOR $d4$
   $C4 = P4$ XOR $d2$ XOR $d3$ XOR $d4$
6. Compute error Position = $C_4C_2C_1$.

```
def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
error_pos = c1*1 + c2*2 + c4*4  # syndrome:
-> faulty bit position.

if error_pos != 0:
    print(f" Error detected at bit position:
{error_pos} -> Correcting...")
    h[error_pos-1] ^= 1          # flip the
    faulty bit
else:
    print(" No error detected.")
    return error_pos
def to_str(bits):
    return " ".join(str(b) for b in bits)
def main():
    print("=== HAMMING CODE - ERROR DETECTION
    AND CORRECTION ===\n")
    n = int(input("Enter number of 4 bit data
    blocks in the stream: "))
    for b in range(1, n+1):
        print(f"\n--- Block {b} ---")
        d = list(map(int, input("Enter 4 data bits
(Space Seperated, e.g 1 0 1 1): ").split()))
        h = encode(d)
        print(f"Original Data Bits :
{to_str(d)}")
        print(f"Encode Hamming Code (7 bits):
{to_str(h)}")

    pos = int(input("Enter bit position to introduce
    error (1-7, 0 = no error): "))
```

```
if 1 <= pos <= 7:
    h[pos-1] ^= 1
    print(f"Transmitted Code (with error induced at pos {pos}): {to_str(h)}")
else:
    print(f"Transmitted Code (no error induced): {to_str(h)}")
print(f"Received Code : {to_str(h)}")
print("checking for error...")
detect_and_correct(h)
print(f"Corrected Code : {to_str(h)}")
rd = [h[2], h[4], h[5], h[6]]
print(f"Recovered Data Bits : {to_str(rd)}")

if __name__ == "__main__":
    main()
```

OUTPUT

```
=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===
Enter number of 4 bit data blocks in the stream: 2

--- Block 1 ---
Enter 4 data bits (space seperated, e.g. 1 0 1 1): 1 0 1 1
Original Data Bits : 1011
Encoded Hamming code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0=no error): 3
Transmitted Code (with error induced at pos 3): 0100011
Received code : 0100011
Checking for errors...
    Error detected at bit position: 3 -> Correcting...
Corrected Code : 0110011
Recovered Data Bits : 1011
```

H

DOMS | Page No.
Date / /

Block 2

Enter 4 data bits (Space Separated, e.g. 1011): 0 1 1 0

Original Data Bits : 0110
Encoded Hamming Code (7 bits): 1100110
Enter bit position to introduce error (1-7, 0=no error): 6
Transmitted code (with error induced at pos 6): 1100100

Received Code : 1100100
Checking for errors...
Error detected at bit position: 6 -> Correcting...
Corrected code : 1100110
Recovered Data Bits : 0110

R

RESULT:

The program to implement error detection and correction using the Hamming code concept was successfully written in Python, executed and verified using a sample input data stream.

Ex.NO:8 NMAP to discover Live Hosts using Nmap Scan (ARP, ICMP, TCP /UDP) on TryHackme platform.

Aim:

To discover live hosts using Nmap scans such as ARP, ICMP, TCP & UDP scans on the TryHackme platform.

Introduction:

Nmap (Network Mapper) is a network scanning & host discovery tool. It is used to identify active systems in a network & gather information about the available hosts and services. Nmap can use different techniques to discover whether a system is active. Common host discovery techniques include ARP scan, ICMP scan & TCP/UDP ping scan.

Host Discovery Methods:

i) ARP Scan:-

Arp (Address resolution protocol) is mainly used to discover host in the local network. An ARP scan sends ARP Request to the same local netw

ii) ICMP:-

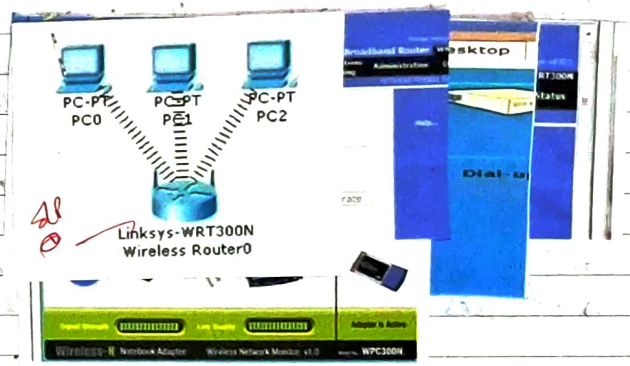
ICMP scan is commonly used to check whether a host is reachable. An ICMP scan sen ICMP requests to the target host & checks for a response. If the host responds it can be identified as a live host.

iii) TCP/UDP scan:

TCP/UDP packets can also be used for host discovery NMAP sends suitable TCP or UDP packets to the target and analyses the response to determine whether the host is active.

Procedure:

* Open the try Hackme Nmap environment.
* Identify the target IP address or IP address range to be scanned.
* Perform TCP/UDP ping scans to discover hosts that respond to TCP or UDP packets.
* Observe the responses received from the target systems.
* Identify the hosts that are currently active or live
* Record the discovered addresses (IP) and scan results
* Compare the results obtained using the different scanning methods.



Conclusion:

Thus the live hosts in the target network were successfully discovered using Nmap ARP, ICMP & TCP/UDP host discovery scans.

Ex: No: 9 Implementation of SUBNETTING in cisco packet Tracer Simulator.

Aim:

To implement subnetting in cisco packet Tracer design multiple subnet with a suitable number of hosts, assign static with IP address & establish communication between different subnets using a router.

Introduction:

Subnetting is a process of dividing a larger network into smaller logical networks called Subnets. Each subnet can contain a suitable number of hosts. Subnetting helps in organizing networks and allows different groups of hosts to communicate through a router.

In this experiment, multiple subnets are designed using cisco packet Tracer. Static IP addresses are assigned to hosts & router interfaces.

The different subnets are then connected through a router, & packet transmission is observed.

Requirements:

* cisco packet tracer
* PCs / Hosts.
* Router.
* Network connections.
* Suitable IP addresses

Procedure:

Step 1: Create the Network

1- Open cisco packet Tracer.

2. Place the required number of PCs / hosts in the

3. Pla

4. Di

on the

5. Conn

Step 2 :

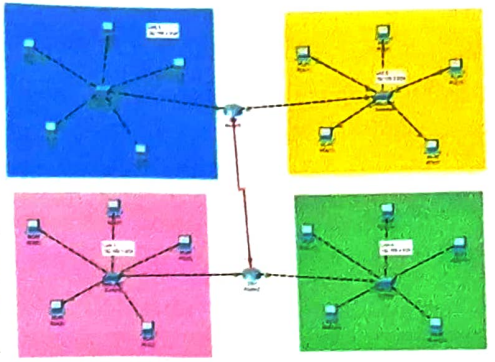
De

1- C

requires

2 Se

3. Di



Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num
	Successful	PC4(2)	Router2	ICMP		0.000	N	12
	Successful	PC4(2)	PC2(1)(1)	ICMP		0.000	N	13
	Successful	PC0	Router0	ICMP		0.000	N	14

smaller subnetwork.

4. Ensure that each subnet has sufficient IP addresses for its hosts

5. Identify the network address and, host address.

Step 3: Connect the hosts belonging to each subnet using the required network connections

Step 4: Assign suitable static IP addresses and masks

Step 5: Configure the appropriate default gateway.

Step 6: Connect the different subnets using router.

Step 7: Configure the router interfaces with IP addresses

Step 8: Check the IP configuration and connectivity of all devices.

Step 9: Perform packet transmission between hosts

Step 10: Use simulation Mode to observe the packet

Step 11: Test packet communication when subnets are connected

Step 12: Test communication without connecting subnets

Step 13: Verify the router is required for communication

Result:

Thus multiple subnets were successfully designed & implemented using cisco packet tracer and communication between different subnets was verified through a router.

DOMS Page No. Date / /

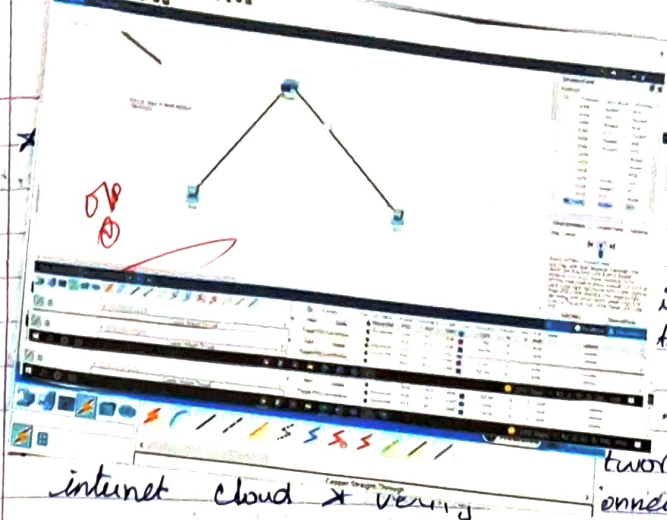
Ex: No: 10 Internetworking with Routers in cisco packet Tracer simulator

Aim:

To design & configure an internetwork using routers in cisco packet Tracer, connect different networks, assign IP Addresses and observe packets transmission.

Procedure:

* Open cisco packet Tracer and create required network topology.
* Create different networks with 3-4 hosts in each network.
* Connect the different networks using a router
* Connect the hosts in each network using the network devices.
* Assign suitable static IP addresses and subnet masks to all machines.
* Configure the appropiate default gateway for each host.
* Assign suitable IP Addresses to the router interfaces connected to the different networks
* Activate and Verify the required router interfaces
* Check the IP configuration and connectivity.
* Perform packet transmission between hosts belonging to different networks.
* Use simulation Mode to observe and trace the path followed by the packet through the router.
* Observe how the router receives and forwards the packets from one network to another.



* using
* wireless
* tomatically
* twork to the
* connectivity.

internet cloud * Verify

* Design another internetwork using a switch router or Ethernet cables.
* Connect the required hosts to the switch and connect the switch to the router.
* Configure the required IP address and router interfaces.
* Perform packet transmisson & use simulation mode to observe the packet flow.
* Verify that Communication is successfully established between the different networks.

Results:

Thus an internetwork was successfully designed and configured using routers in cisco packet Tracer. Different networks were connected using routers, Static IP address were assigned, packet routing was observed using wireless router, DHCP, internet cloud switch & Ethernet cables was successfully verified.

Ex. No: 11 Routing a Network layer

a. Simulate Static Routing Protocol Configuration

b. Simulate RIP

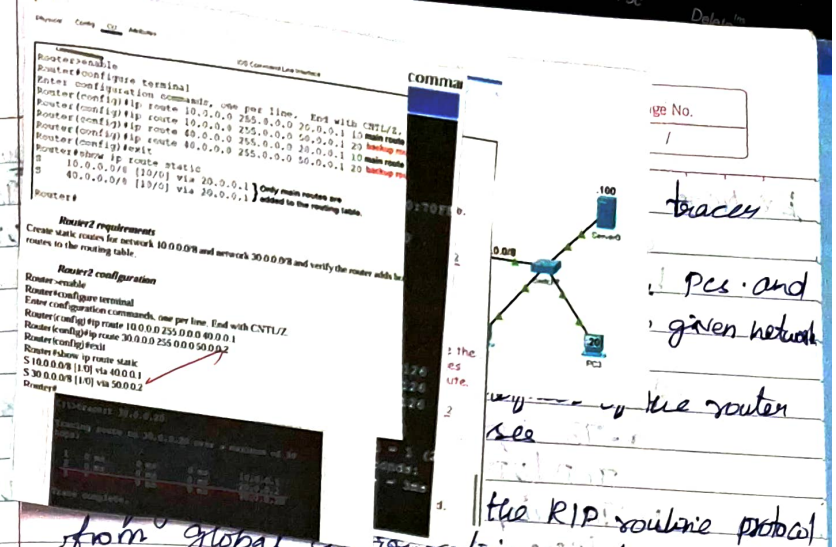
Aim:

To Simulate Static Routing protocol configuration and RIP using Cisco packet Tracer

Algorithm / Procedure:

Part - A. (Static Routing):

* Open Cisco packet Tracer and create the given network topology with routers
* Configure the networks available on interfaces
* Open the router CLI and enter the required configuration mode.
* Configure the first route as the main route and the second route as the backup route
* Use the tracer command to trace the path taken by the packets.
* To test the backup route, remove the link between the routers and observe the change in the routing path.
* Configure the backup static route and verify that packet is transmitted through the alternate route.



traces

Pcs and

given network

the router

see

the RIP routing protocol

from global configuration mode.

* Specify the directly connected networks that have to be advertised using RIP.
* Use the ping command to test connectivity between the Pcs.
* Use the tracer to command to observe the route followed by the packets
* If the selected route fails, RIP automatically selects another available route for packet transmission

Result:

Thus static routing & RIP were successfully simulated using cisco packet tracer and packet transmission and alternate routing were verified.

Ex. NO:12 End - End Communication at transport Layer.

a. Implement 'Echo Client Server Using Tcp/UDP Sockets

b. Implement chat client server Using TCP/UDP sockets

Aim:

a. To implement echo Client Server, Using TCP/UDP sockets.

b. To implement chat Client Server Using TCP/UDP sockets

Algorithm:

Part A. Echo Client Server.

* Implement an echo client - server Using socket programming.

* Use TCP/UDP sockets for communication between the Client & Server.

* Establish Client - Server communication through the transport layer

* Exchange data/message between the client and server.

* Record the input and output obtained during the Communication.

Part B

Chat client Server:

* Implement a chat client server using socket programming.

* Use tcp/UDP sockets for communication between the client and server.

* Establish communication between the client and server.

* Exchange messages between the client and server.

* Observe the message received at both the client and server sides.

Input:

Part A:

Message/data given by the client for echo communication

Part B:

Message exchanged by the client and server during chat communication

Result:

Thus Echo - client server & chat Client Server communication using TCP/UDP Sockets were successfully implemented.

Ex: NO: 13 IMPLEMENT YOUR OWN PING PROGRAM:

Algorithm:

1. Start the program.
2. Import the required socket and system libraries
3. Get the destination IP address or hostname from the user.
4. Create a an ICMP socket to send ping requests
5. Construct an ICMP Echo Request packet.
6. Send the packet to the destination host.
7. Wait for an ICMP Echo Reply from the destination & calculate the round-trip time. If a reply is received
9. Display the response time or a timeout number of packets.
10. Repeat the process for the required number of packets.
11. Close the socket and terminate the program input

Enter the destination IP address: 8.8.8.8
Enter the number of packets: 4

Output:

Pinging 8.8.8.8

Reply from 8.8.8.8: time = 25 ms
Reply from 8.8.8.8 time = 24 ms
Reply from 8.8.8.8 time = 26 ms
Reply from 8.8.8.8 time = 25 ms.

Ping operation completed.

Result:

Thus Ping operation is completed Successfully.

Ex: NO: 14 Packet Sniffing Using RAW Sockets:

Algorithm:

1. Start the program.
2. Import the required socket library.
3. Create a raw socket using the appropriate protocol.
4. Bind the socket to the required network interface if necessary.
5. Capture incoming packets using the raw socket.
6. Read the captured packet data in byte format.
7. Extract and display the IP header information such as source IP address, destination IP address, and protocol type.
8. Continue capturing packets until the user stops the program.
9. close the socket and terminate the program. input.

Network Interface: Active network interface
Packet Capture: Start.

Output:

Packet Sniffing Started...

Source IP Address: 192.168.1.10
Destination IP Address: 142.250.182.14
Protocol : TCP

Source IP Address: 192.168.1.10
Destination IP Address: 142.250.183 8.8.8.8
Protocol: ICMP.

Result:

Packet Sniffing completed.

Ex:No.15 WEB LOG ANALYSIS USING WEBALIZER

Aim Diagram:

Input:

Logfiles:

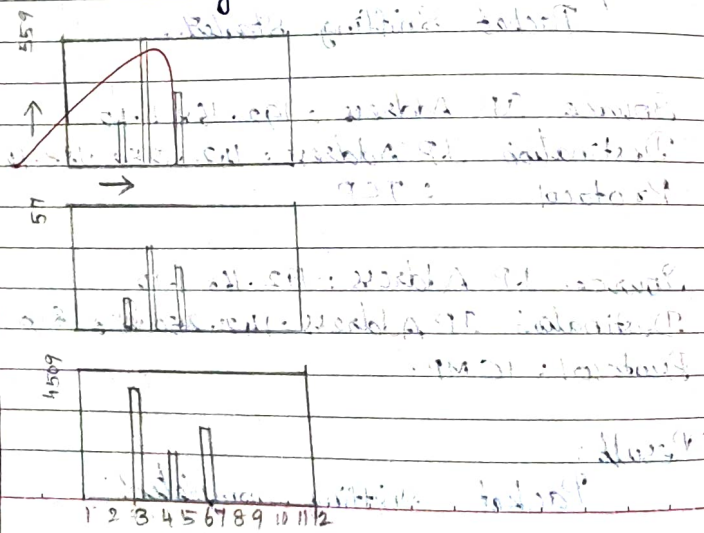
C:\Users\ICS\Downloads\access.log

Target Directory:

C:\Users\ICS\

Output:

Monthly Statics:



Ex:No.15 WEB LOG Analysis Using Webalizer.

Aim:

To analyse different types of users logs using the Webalizer tool.

Procedures:

1. Run the Webalizer windows version.
2. Input the usebar log file downloaded from the web
3. Press Run webalizer
4. Webalizer processes the log file and generates statistical reports.
5. View the generated report in a web browser.
6. Webalizer reads and processess the log file
7. It analyses the requests and visitor information
8. The tool generates statical reports and graphs
9. Open the generated report on a web browser.
10. Analyse the number of visits, pages and requests, bandwidth usage.

Result:

Thus, the different types of web logs were successfully analyzed using the webalizer tool.

DOMS Page No.
Date / /